Combining Moodle Course Manager With Application-Specific ChatBots And Open-Access Libraries On Generic Headless Linux Computers To Create Portable Wireless Information Centers For Remote Situations

Peter G. Raeth
https://github.com/SoothingMist

This document and the associated files describe an exploration of technological capability that resulted in a proof-of-concept which has not been tested in an industrial environment. All project files can be downloaded from https://github.com/SoothingMist/Portable-Wireless-Host. Use at your own risk.

Table of Contents

Introduction

Portable information centers hosted by headless computers create cost-effective, rapidly deployable, and flexible support for classrooms, labs, community libraries, and other application-specific requirements. They enable adaptation to situations where internet is unavailable, inconsistent, or not cost-effective. They can also add to existing resources while not requiring the purchase of significant and expensive hardware and software. Such centers operate wirelessly and are accessible by smart phones, tablets, and laptops. They can also be powered by battery and other sources of electricity. This creates new learning, teaching, and other operational opportunities for remote and rural environments.

The resulting wireless server needs no access to the world-wide web but it creates its own internet, complete with open-access material and services as added by a teacher, content creator, or librarian. Important aspects of this project include:

- Offline Access: Provides inexpensive access to information and software in areas without reliable and cost-effective internet.
- Purpose-Associated Library and Software: Pre-installed content can be added as needed to suit particular circumstances, teaching requirements, or operational realities. Content and software can be tailored for specific needs, such as schools, clinics, remote communities, or particular tasks.
- Low Cost Generic Hardware: Can operate on cheap battery-powered hardware, including a relatively inexpensive RaspberryPi and other such small compact computers that fit in one's pocket. Also can be installed on larger wireless hosting hardware. The project descripted here is designed for generic computers running the Debian or Ubuntu versions of the Linux operating system.

Beyond information and software associated with a given activity, it is also possible to host task-specific chatbots on the very same hardware and environment. Everything can be accessed wirelessly and simultaneously via the same browser. Although slow and not intended for time-critical responses when run on such computers as RaspberryPi, the chatbot can be used to query documents within a task's domain. For example, one could use a textbook or other material to improve a chatbot's ability to provide additional means of study. The textbook itself could be available through the same interface. Yes, copyrighted texts cannot be used in this context but there are numerous high-quality textbooks available through open-access sources. (Search some of these for examples.) A task-specific library can also be included.

One may ask why the author chose to not employ Internet-in-a-Box (IIAB) or MoodleBox. Both are well-respected and aim at the same purpose as this project. Moodle itself is very popular for learning management. Speaking for himself alone, the author was unable to get Moodle operating reliably under IIAB. Nor was he able to get IIAB and Moodle to install as independent processes. MoodleBox is designed for the RaspberryPi, installing the RPi OS and Moodle so that the Moodle learning management system is accessible wirelessly while RPi acts as the host. It was possible to get IIAB *or* Moodle, plus the project's chatbot and library to operate independently but not IIAB *and* Moodle. So, the author decided to use IIAB only as one means of identifying sources of open-access content.

As one bright person said, "Installing content and services for remote access on a stand-alone host is not for the weak of heart." It is also not for those impatient with details. What follows are the steps we took to host a portable information center on a headless wireless-capable resource-limited computer running the Debian-Linux operating system. This device need not have access to the world-wide web but acts as a stand-alone internet.

Hardware

This project is designed to run on minimal hardware. The RaspberryPi chosen for this project was already on hand. It serves as a flexible platform for various applications. Keep in mind that the RPi is just a small resource-limited computer running either the Debian or Ubuntu version of the Linux operating system. It does have certain hardware capabilities that are not used in this project.

Parts List and Associated Equipment

- RaspberryPi with enclosure (not waterproof)
- MicroSD card (256gb minimum)
- MicroSD card reader/writer (needed only for configuration)
- Ethernet connector cable (not needed once RPi is configured)

The author carried out the described installation/configuration using a Windows 11 computer connected to the RaspberryPi via Ethernet. Once completed, the battery-powered headless RaspberryPi operates wirelessly and independently.

Setting up the RaspberryPi

Here are the steps followed to get RaspberryPi ready to use. This device operates just like any headless Debian-Linux device. ("Headless" refers to there being no display nor keyboard. Installing Debian-Linux on some other device requires different steps.)

1. Download, install, and run the RaspberryPi OS imager. This step requires the use of the MicroSD card and the card reader/writer. During this process, be sure to enable SSH and WiFi. Use the Other OS option then the 64-bit Lite option. The burn takes a few minutes and goes smoothly. Be careful when handling the MicroSD card. It is very small and will not tolerate rough treatment. Keep fingers away from its exposed electrical contacts lest you foul them.

2. Dismount the SD card from the reader/writer and mount it on the RPi.

3. Connect the RPi to your local network and plug the RPi into a power source. Which socket to use on the board is clearly marked. It does not appear possible to plug the power supply into the wrong socket. Plug the power supply's adaptor firmly into the socket. If you do it too lightly, it will not connect. However, as in all such things, if you have to force it, you are doing something wrong. The power supply's switch is not marked for on/off. For those with good eyes in good lighting, there appears to be a nub on the switch that marks the ON position. If you press the switch down on that nub, in the direction of the device, the power is on. Once the device is booted, a red LED will show. The green blinking LED will go out.

4. Check for presence on your network. Your router's device-list should show the name you gave your device, and its address. Bring up your PC's command console. Enter the command *ping <device name>*. Ping should confirm the device's presence. Use Putty and Port 22 to connect to your RPi. A login console will appear. Entered the username/password you established during the burn process. A Linux prompt will appear. *python --version* shows the latest python language version is installed. That is sufficient for our purposes. (If you can see the RPi on your network but still cannot connect, see the Appendix dealing with a more complex approach to burning the OS to the RPi and configuring Putty.)

5. At this point, it is best to fully update the version of Debian Linux presently on the RPi. Here are the steps used by the author. These take a few minutes to run.

   sudo apt update
   sudo apt upgrade -y (Recommend answering "Y" to subquestions.)
   sudo apt full-upgrade -y
   sudo apt autoremove -y
   sudo apt clean

6. Once the device's operation is verified and updated, install the RPi into its case. Use *sudo shotdown now* to turn off the device. Then set its power switch to off. Disconnect the power supply and Ethernet cable. Remove the SD Card from the device. Otherwise, it is likely the card will break. Follow the instructions that come with the case. All parts and the screwdriver are provided in the kit. Note: Pins on the device are not numbered but Pin-10 is marked. Look carefully at the diagram in the instructions. Count pins carefully when connecting the fan. After Step 2 in the instructions, notice the two heat sinks provided in the kit. There is no instruction regarding the heat sinks but have a look at this video. This video shows how to install the provided heat sinks. These appear meant for the processor and memory chips. Heat sinks for the USB and Ethernet controllers are not included in the kit. Before Step 3 in the instructions, install the two provided heat sinks. After Step 5 in the instructions, the two halves of the case are not tight together. There is still looseness. More tightness applied to the screws risks stripping the screws' bodies and heads. (If the loose case is a concern, perhaps a plastic tie strap would fix that.) Once Step 5 is completed, notice the back of the case, the opposite side from the fan. There are two wall mounts. If you mount the RPi to something, be careful of how long the mounting screws are, and the size of the heads. It is possible to damage the board or short it out. The case is not designed to accommodate pin-mounted plugins. Notice too the four round indentations on the back of the case. These are for the feet provided in the same bag with the screws. A scissor serves to cut these to size for mounting. The material surrounding the round feet is designed to peal away. Installing the feet and laying the case down on the feet improves air circulation and thus cooling for both wall and table mounting. Finally, notice the carry-case that comes with the kit. This is a nice travel and storage case.

7. Verify installation and operation. Reinsert the SD Card. Reattach the power supply and Ethernet cable. Turn the power switch to ON. The fan should come on. The red and Ethernet LEDs should also come on. Use Putty to connect to the device and log in. "python --version" should show that the very latest version is installed. This completes installation, configuration, and verification of the RPi.

Setting up a Hotspot

After getting your device loaded with the Debian or Ubuntu OS, the next step is to set it up as a remote host that is independent of any network. This is not to deny access to the world-wide web but we assume that is not available and so will be hosting all content and services on the device (a RaspberryPi in our case). We still have to access the device to add content, load software, and adjust configurations. We will be able to do that via Ethernet. There is an Ethernet port on the RPi so we will use that via Putty.

A tutorial shows how to set up a device to act as a hotspot offering access to local internet entry points. We apply the first part of the tutorial, which shows how to set up as a hotspot without world-wide web access. Follow those steps until you get to the section "Configure

Connection Portal". Once you finish all that, your hotspot will be visible under "Available Networks". The material after that would enable you to connect to a world-wide web access point if you add an external USB Wi-Fi adapter.

One the basic hotspot is set up, open the terminal to your device. Run this command to see the IP of the hotspot*, ip route show | grep wlan0*. Look for the IP address right after the word *src*. For example, if it says src 10.42.0.1, then 10.42.0.1 is your host's exact address. Keep that in mind as we add content and services.

Demonstrating a Simple Webpage

In this section we set up a simple webpage for wireless access, the full code for which is in the directory SimpleWebpage. Start by opening the terminal to your device. Then create a directory to hold your various content and services. We are going to be using programs written in Python so we could name the directory PythonServices. Within that directory, we need to create a Python virtual environment:

```
sudo apt update
sudo apt install python3-venv
python3 -m venv venv
```

Then we activate Python's virtual environment with

```
source venv/bin/activate
```

We need to be able to install Python modules

```
pip install --upgrade pip
```

The virtual environment can be deactivated when no longer needed

```
deactivate
```

Each content or service goes in its own sub-directory. For this demonstration, lets use SimpleWebpage. The program that drives the demonstration requires certain Python modules. To identify those, we create a file, requirements.txt. Into that file we place the name, *flask*. As we move forward, that file can be copied and module names added to it. Lets download that module, *pip install -r requirements.txt*. We now create the program file, main.py with the following code:

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route('/')
def home():
  return render_template('index.html')

if __name__ == '__main__':
  app.run(host='0.0.0.0', port=5000, debug=True)
```

Notice the use of port 5000. Developers most commonly use ports 5000 through 5100 and 8000 through 8090 sub-ranges for Python Flask deployments. The program reads the webpage and serves it. The webpage itself goes into its own sub-directory, templates. In that sub-directory, add this html code to index.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Flask Webpage</title>
</head>
<body>
  <h1>Hello from Flask!</h1>
  <p>This simple webpage is being served on port 5000.</p>
</body>
</html>
```

Once that is completed, you should have the following directory/file structure:

```
PythonServices
        venv
        SimpleWebpage
                main.py
                requirements.txt
                templates
                        index.html
```

To start the service, *python main.py*. Something like the following will appear:

```
 * Serving Flask app 'main'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.1.222:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 733-068-954
```

The use of a development server is fine as we are only running a locally-accessible host that is not connected to the world-wide web. In a sense, we have created our own local internet that is completely disassociated from all other networks.

Now activate whatever wireless-access device you like. Connect to your host. Bring up the webbrowser. Go to this address, http://<your host's IP address>:5000 (in my case, http:// 10.42.0.1:5000). If all has been done correctly, your browser will show:

> Hello from Flask!
>
> This simple webpage is being served on port 5000.

Setting a Different Host IP Address

It is worth noting that the IP address 10.42.0.1 was assigned by default when the host was first configured. One can imagine needing more than one host. If that is the case, a different unique address should be manually assigned and all other configuration settings that reference that address adjusted accordingly. For example:

```
sudo nmcli connection modify "Hotspot" ipv4.addresses "10.43.0.1/24"
sudo nmcli connection down "Hotspot"
sudo nmcli connection up "Hotspot"
```

Establishing an Always-On Service

At issue with our initial success is that we have to manually start the service by accessing the headless computer somehow. Would it not be better to have the service run as a process that starts as soon as the host boots? That certainly is possible. Lets see how.

Put the following file, GenericPython.service, into the PythonServices directory:

```
[Unit]
Description=<short descriptive text>
After=network.target <uniqueName>.service

[Service]
Type=simple
User=<result of whoami command>
WorkingDirectory=<full path to project directory, result of pwd command>
ExecStart= bash -c ' cd <full path to project directory> && source
 ../venv/bin/activate && python main.py'
Restart=always

[Install]
WantedBy=multi-user.target
```

This is a generic description of an always-on service. In the author's case, he placed the following in SimpleWebpage.service within the SimpleWebpage directory:

```
[Unit]
Description=Demonstrates a simple webpage as a service
After=network.target SimpleWebpage.service

[Service]
Type=simple
User=peter
WorkingDirectory=/home/peter/PythonServices/SimpleWebpage
ExecStart= bash -c ' cd /home/peter/PythonServices/SimpleWebpage && source
   ../venv/bin/activate && python main.py'
Restart=always

[Install]
WantedBy=multi-user.target
```

Copy SimpleWebpage.service to /etc/systemd/system. Use the following commands to start the process and check its status. After the process is enabled, it will start every time the host boots.

sudo systemctl daemon-reload
sudo systemctl enable --now SimpleWebpage.*service*
sudo systemctl status SimpleWebpage.*service*

If the status report shows errors, use this command to help trace down the cause:

sudo journalctl -u SimpleWebpage.service -e

Once the status shows success, reboot the host. After the reboot process finishes, you can check further by bringing up the service in your wireless browser. It will be running on the same address that was used earlier. To stop the chatbot from running, use *sudo systemctl stop* SimpleWebpage.*service*.

Thus we have the creation and demonstration of an independent web host. Now we can add other services and content.

Installing Moodle – A Learning Management System

Moodle is a popular and effective learning management system. We followed the installation guidelines for v5.2. Very informative were the two postings by Farhan Waheed Khan (text, video). Google also helped a great deal (Ex: For Moodle v5.2, how to …).

Our intention is to only show how to install Moodle, not how to manage it or employ it. Also, some configuration variables for Moodle itself or for its dependencies will be different from those proposed in other guidance. These settings were necessary in the experience of the author.

The installation process starts by installing dependencies.

Installing Apache Webserver

The first need is a webserver. Apache is the one chosen for this project because it is well-known and well-documented.

sudo apt install apache2 -y
sudo systemctl reload apache2
sudo systemctl restart apache2
sudo systemctl status apache2

After Apache is installed, You should now be able to use your browser to access Apache's default webpage at http://<your device's address>/index.html.

Installing a Database Management System

Once Apache is verified, we next need a database management system. MariaDB, a variant of MySQL, is the one used in this project because of its versatility.

```
sudo apt-get install -y mariadb-server mariadb-client
sudo systemctl status mariadb
```

Now configure the database for use by Moodle. You can choose username and password to whatever you like. Just remember what you chose.

```
sudo mysql -u root -p
    CREATE DATABASE moodle DEFAULT CHARACTER SET utf8mb4 COLLATE
   utf8mb4_unicode_ci;
    CREATE USER 'moodleuser'@'localhost' IDENTIFIED BY 'YourPassword123!';
    GRANT ALL PRIVILEGES ON moodle.* TO 'moodleuser'@'localhost';
    FLUSH PRIVILEGES;
    EXIT;
```

Also edit /etc/mysql/mariadb.conf.d/50-server.cnf so that max_allowed_packet is set to at least 128M (1G is fine). This is needed for restoring Moodle courses.

Installing PHP

Needed next is PHP and some of its extensions and libraries. (sodium and openssl were already included in the author's install of php. You can check if an extension or library is loaded using: php -m | grep <extension/library name>).

sudo apt install php -y sudo apt install php-cli -y sudo apt install php-ctype -y sudo apt install php-curl -y sudo apt install php-dom -y sudo apt install php-gd -y sudo apt install php-iconv -y sudo apt install php-intl -y sudo apt install php-json -y sudo apt install php-mbstring -y sudo apt install php-simplexml -y	sudo apt install php-xml -y sudo apt install php-zip -y sudo apt install php-mysql -y sudo apt install php-common -y sudo apt install php-mbstring -y sudo apt install php-xmlrpc -y sudo apt install php-soap -y sudo apt install php-tokenizer -y

PHP has to be configured so that it supports Moodle.

```
sudo nano /etc/php/8.4/apache2/php.ini
    memory_limit = 1024M
    upload_max_filesize = 64M
    post_max_size = 64M
    max_execution_time = 3600
    max_input_vars = 10000
```

Installing Moodle Itself

Now that dependencies are installed, it is time to install git and Moodle itself.

```
cd /var/www/html
sudo apt install git -y
sudo git clone -b MOODLE_502_STABLE git://git.moodle.org/moodle.git
```

Then configure the file structure.

```
sudo mkdir /var/moodledata
sudo chown -R www-data:www-data /var/moodledata
sudo chmod -R 777 /var/moodledata
sudo chown -R www-data:www-data /var/www/html/moodle
sudo chmod -R 755 /var/www/html/moodle
```

Apache, the web server, has to be configured to see Moodle as its baseline website.

```
sudo nano /etc/apache2/sites-available/moodle.conf
    <VirtualHost *:80>
      ServerAdmin put administrator's email address here
      DocumentRoot /var/www/html/moodle/public
      ServerName localhost

      <Directory /var/www/html/moodle/public>
        Options FollowSymLinks
        AllowOverride All
        Require all granted
      </Directory>

      ErrorLog ${APACHE_LOG_DIR}/error.log
      CustomLog ${APACHE_LOG_DIR}/access.log combined
    </VirtualHost>
```

```
sudo a2dissite 000-default.conf
sudo a2ensite moodle.conf
sudo a2enmod rewrite
sudo systemctl reload apache2
sudo systemctl restart apache2
```

Moodle's configuration has to be initialized. Copy /var/www/html/moodle/config-dist.php to /var/www/html/moodle/config.php. Then, set /var/www/html/moodle/config.php variables:

```
$CFG->dbtype    = 'mariadb';
$$CFG->dbuser   = 'moodleuser';
$CFG->dbpass    = 'YourPassword123!';
$CFG->dataroot  = '/var/moodledata';
$CFG->wwwroot  = 'http://<your host's address>'
```

At this point, everything should be ready. I prefer to reboot*: sudo reboot*. After reboot completes, use your browser to access Moodle: http://<your host's address>. Once the page loads, answer the various questions. You will notice a page with Server Checks and Other Checks. As long as all Server Checks are positive, you are fine, your installation meets all minimum requirements.

Setting Browser-Level Moodle Configuration

Because of the other configuration settings we made, it is necessary to set certain configuration values from Moodle's administrator webpage.

Under Site administration > Server, set Extra PHP memory limit to 1024. Also set Maximum time limit to 3300. Press Save changes at the bottom of the page.

Under Site administration > Courses > Asynchronous backup/restore, uncheck Enable asynchronous backups. Press Save changes at the bottom of the page. This means you have to stay on the restore page until the course restoration is completed. The author has found that is best overall.

Adding a Course to Moodle

To illustrate a course within the Moodle framework, The author went to the open-access course repository hosted by OER Commons. They are the only actively-maintained publicly-accessible Open Education Resources (OER) repository he could find. "The worldwide OER movement is rooted in the human right to access high-quality education. This shift in educational practice is not just about cost savings and easy access to openly licensed content; it's about participation and co-creation. Open Educational Resources (OER) offer opportunities for systemic change in teaching and learning content through

engaging educators in new participatory processes and effective technologies for engaging with learning." They are not just for K-12. You will find college courses here as well. Unfortunately, one has to search carefully for those uploadable to Moodle. Type *Moodle* in the *What are you looking for* entry and press <return> to find a few of those. To demonstrate adding an existing course, I chose a physics lesson on optics by Carsten Lenze in Germany. Download the mbz file for that lesson.

At the Moodle administrator's page, go to the "Site Administration" tab and then to the "Courses" tab. Click on the "Restore Course" link. Drag and drop the selected course mbz file into the resulting box. Stay on that page to continue with Moodle's restore process until it completes. The selected course contains numerous pdf files as readings in English. The author chose to keep those and remove everything else.

Moodle has numerous ways to create and access a course, and numerous ways to add materials and activities to a course. See their documentation for that and to the many postings on course design. This is a vast domain that is outside this project's scope. We only try to show how to make the capability available wirelessly under circumstances where there is no access to the world-wide-web. For our immediate purposes, we enabled guest review of courses. You can allow guests to see all course materials but without actually enrolling in courses.

Something the author had difficulty with is allowing guests to actually see course materials for courses that are enabled for guest viewing. There was also a problem with allowing guests to properly logout.

As Administrator

Site administration > Appearance > Default dashboard

Enable Edit mode

Add a block > Text

Into that text box, put in text and links for

Course List: http://<device address>/course/index.php
Logout: http://<device address>/login/logout.php

Disable Edit mode

Verifying Wireless Connection

At this point, it is worth testing wireless connectivity. All wireless-capable devices have the ability to list the wireless hosts with which they could potentially connect. How to access that list depends on the device you are using. After connecting, you may be asked

to give a password. That password was determined when the host was first configured. After connecting, open your browser and connect to http://<host address>. Moodle's login screen will appear. Proceed from there according to your Moodle-access credentials.

There will be times when it seems all configuration settings are correct but connection still cannot be achieved. This could be a browser problem, in which case clearing the browser's cache is a good ides. Another issue could be Moodle's cache. That too needs to be purged at times. To do this, use:

```
sudo -u www-data php /var/www/html/moodle/admin/cli/purge_caches.php
```

Generating an Application-Specific ChatBot

One may wish to add a chatbot to support a given application. Since the author is not an expert in chatbots and their underlying technology, he used the industrial chatbot Grok as a means of learning from established knowledge. While the result is not integrated with Moodle, it runs on the same host at the same time using a different browser tab and a different host port (http://<host address>:<port>). Links external to Moodle can be provided within Moodle so that they open a new browser tab. The same is true of any webpage.

This Grok interaction thread makes interesting reading and shows how the chatbot session evolved over several days. Only the very first code listing worked as presented in the thread. The rest had to be modified in various ways. In the end, one has to realize that copy/paste is never going to work. It is necessary to have a solid background and engage in additional study. Grok is just pointing in a particular direction, while offering references for further reading. The following material discusses the final result. Code files are available on the GitHub project page. You need familiarity with the python programming language.

Adding Documents to a Baseline Large Language Model

Building an application-specific chatbot starts with a Large Language Model (LLM). There are numerous LLMs in existence. We focus on models with one billion (1B) or fewer parameters because the chatbot is intended to run on resource-limited computers. While larger models may load, the delay between query and response is far too long. The same will happen if too many documents are added during the Retrieval-Augmented Generation (RAG) process.

Download and Install Ollama

Ollama is an open-access tool for managing open-access LLMs. Download and install Ollama on a full-up computer (one having a GPU) and on the computer that will serve as a wireless host. We chose a RaspberryPi, a small resource-limited computer running Debian

Linux. We set all that up before we installed Moodle. You can test ollama's installation with *ollama --version*.

Download and Install an LLM

The LLM we will use is well known and respected, gemma3:1b, a lightweight model developed at Google. It is designed for high-efficiency and resource-limited applications. Its main features are:

- Excellent for on-device tasks such as Q&A, summarizing, reasoning, and generating text from images.
- Capable of high-throughput performance, designed for fast inference on consumer CPUs, GPUs, and mobile devices.
- Supports over 140 languages.

gemma3:1b is easy to install. At the console command line use *ollama pull gemma3:1b*. To verify installation use *ollama list*. Reasonable alternative models are phi3:mini or llama3.2:1b. For something even smaller, you can try gemma2:2b or tinyllama. You can query the model as-is: *ollama run gemma3:1b*. Type */bye* at the prompt when you are satisfied.

Carrying out the RAG Process

Once we have ollama and gemma3:1b installed, we want to add documents to the model's information collection using our full-up computer. There are numerous books available via open-access that can be used for this purpose. Take some time to decide on an application and then explore the world's collection of open-access books.

For the first test we downloaded the Boy Scout manual of the early 1900s, along with a couple of related books. The documents are on the GitHub page. Create a project directory and put those files in the subdirectory *rag-data*. Of course, you can use whatever books you like. Most common formats are supported.

The files for carrying out our first RAG process can be downloaded from the following sources:

- Boy Scouts Handbook : The First Edition, 1911
- Knots, Splices and Rope Work
- **Scouting for Boys**

These go in *rag-data*, a sub-directory of RAG_AddDocuments. The RAG process needs to be run on your full-up computer. Tried it on a computer that has no GPU. After a day of

processing it did not finish. On a GPU machine, it took mere minutes. Notice the accompanying file, requirements.txt. This is a list of python library modules that have to be added to your python environment.

We also need the embedding model nomic-embed-text. Use this command to install it: *ollama pull nomic-embed-text*. This is a popular high-performance open-access embedding model with a large token context window. It is often used during RAG processes.

Once the RAG process in the directory RAG_AddDocuments finishes, it will ask about your queries. You can play with it if you like. Interesting general queries are: “What do you know most about?” and “Do you know anything about <topic>?”.

Notice that the RAG process resulted in the sub-directory *storage*. Thus, the RAG results are persistent and can be copied to the host. After copying the subdirectory storage to your host’s project directory. You should also copy QueryModel.py. This file is the same as RAG_AddDocuments.py except that the RAG process itself is not present.

Running QueryModel.py on the host and asking a relatively simple question, “What is the boy scout motto?”, yields the correct answer in about 8 minutes. A more complex question, “We are in a very remote area. There is no medical help nearby. One of our members has suffered a long deep cut. It is bleeding badly. How should we treat the wound?”, yields a correct but basic response in about 9 minutes. (Keep in mind that complex medical data was not employed in the RAG process.) The timeout set within this code is 10 minutes. That can be easily changed.

Clearly, this chatbot arrangement is not suitable for critical , need to know right now, applications. It does offer basic answers to application-specific questions. Still, more than one article asserts that the accuracy of chatbots overall is only around 80%. That may be good enough for a first cut but indicates that all chatbot responses have to be taken with a grain of salt. One cannot blindly follow a chatbot.

Integrating with our Environment

Having demonstrated the basic idea behind application-specific chatbots, we now want to create one that is accessible wirelessly. While we will not integrate our chatbot directly with Moodle, we will install it on the same host and access it on the same wireless network via a port number different from the one Moodle uses. Thus, Moodle and our chatbot can be accessed via the same browser in different tabs.

Notice that different people can access the wireless network and the content/services it hosts, all at the same time. Beware of heavy use as our host has limited resources. If you

want many people to be on at the same time, you may find a host with greater resources to be better. The cost will be greater, especially if more memory or a faster CPU are included.

Like the earlier chatbot demonstration, there are two versions of this code.

- Ollama_New_RAG has a button that conducts the RAG process.
- Ollama_New_Host has no RAG capability but still enables queries.

On the host, copy Ollama_New_Host.py into a new project directory. Copy the sub-directory *storage* from our previous RAG efforts. Then create another sub-directory called *history*. Run Ollama_New_Host.py. That will start the chatbot running wirelessly. Access via http://<host address>:8080. Notice how it gives responses and references back to the original documents. Those documents can be added to the host, if there is enough space left for that. You can check for remaining space using *df -h*. How to make documents available on s wireless host is demonstrated in the next section. To run the chatbot as a service that starts upon boot, create an Ollama_New_Host.service file and configure it, using what we did when we created a generic web service as a guide.

Creating and Hosting a Library

There are various approaches to building a library for remote stand-alone hosting. All approaches must end with actual books/documents/papers/reports/data being stored on the host for remote access. We will illustrate two approaches here.

Manual Generation

The first approach searches a vast collection of internet-hosted books and manually downloading them one by one, while recording their title, abstract, and filename. Perform the search using Open-Access Meta Search. Then record the results of your search.

Within the directory ManualBookLibrary is a sub-directory, static, and within that, Books. Within the Books sub-directory, create a text file, Catalog.txt. (See the example, CatalogExample.txt.) Into that file record the title, abstract, and the filename of your book selections. Use this format:

Title #1
Abstract (all on one line, no line breaks)
Filename (no spaces)
Title #2
Abstract
Filename
... and so on ...

Download your selected books into the Books sub-directory. You can download the books for the example from:

The Wonders of Optics

University Physics - Volume 3

Run main.py and bring up the resulting webpage. Your catalog will have been read and a webpage generated that describes and links to your selected books. If your catalog does not change, you can comment the subroutine call in the main function that generates the webpage. When all is said and done, this should be the directory structure:

```
ManualBookLibrary
        main.py
        requirements.txt
        templates
                index.html
        static
                Books
                        Catalog.txt
                        <books>
```

Automatic Generation

Some open-access repositories do enable the programmatic search and download of individual books. The author has identified two of those: DOAB (Directory of Open-Access Books) and Project Gutenberg. While certain others claim to enable such processes, they either require payment or intrusive registration.

Using the MARC records provided by our two sources, It is possible to generate a database that contains title, author, link. An interface can be created that allows for the automated search, selection, and download of books cited in the database. The author has generated that database and interface. See the directory AutomatedBookLibrary. This interface offers a search capability, the result of which contains check boxes. There is a button that causes the download of any book whose box is checked. While the download process is underway, an X will appear to the left of the webpage's address. When the process is finished, a circle will replace the X and there will be a blank screen where the booklist had appeared.

Merging the two Approaches

Automatic Generation produces a Catalog.txt that can be appended to that produced during Manual Generation. Once appended and the associated book files copied to the Books directory of Manual Generation, that process can be run and an index.html will be

produced that links to all books, whether manually or automatically downloaded. Either way, the index.html generated by the manual approach or the Catalog.html generated by the automatic approach can be used as the index.html for SimpleWebpage to create a remotely-accessible catalog that links directly to on-host books.

A Look at What we Accomplished

We have seen how to use a resource-limited headless computer running the Debian-Linux operating system to create a remote host. This host acts as its own "internet" server without being connected to the world-wide web or any other network.

Through this host, we have shown how to create and access various webpages and services. Taken together, the examples show how course material can be supplemented by a library and chatbot. This generic capability provides a foundation for numerous other applications in remote domains where access to the world-wide web may be unavailable or expensive to attain.

"Remote" is a critical term when one considers how to power such a capability. We explore that in the next section.

Powering Remote Hosts

Google summarizes remote-powering issues in this way:

> *When you are off the grid, the best way to power your devices depends on your energy needs and how long you plan to stay.*
>
> *For short trips or light devices, portable power banks and solar chargers are excellent. For longer stays or running heavier appliances, portable solar generators or fuel-powered generators are the standard. If you are staying long-term, building a dedicated off-grid solar array or using a micro-hydro turbine (if you have a running stream) are sustainable options.*
>
> *Here are the most reliable options broken down by use-case:*
>
> *1. Solar Power*
>
> *Small Solar Chargers: Ideal for keeping phones, GPS units, and small cameras running on day hikes. Check out the REI Solar Chargers Guide for lightweight, portable options.*

Portable Solar Generators: These combine a solar panel with a battery pack, ranging from small 200Wh units (for laptops) to 3000Wh+ units (for running refrigerators and power tools). Explore models and specifications.

Permanent Off-Grid Systems: For cabins or long-term setups, a roof-mounted solar array tied to deep-cycle battery banks (such as Lithium Iron Phosphate) is the most reliable long-term investment.

2. Fuel-Powered Generators

Portable Inverter Generators: These run on gasoline, propane, or dual-fuel, and provide stable power that won't damage sensitive electronics like laptops. They are quieter and more fuel-efficient than traditional frame generators. Find top-rated models via the Consumer Reports Generator Buying Guide.

3. Thermoelectric & Wind

Thermoelectric Generators: These generate electricity directly from heat. Devices like the BioLite CampStove allow you to charge small USB devices while cooking a meal over a wood fire.

Wind Turbines: If you are in a consistently windy, remote area, a small wind turbine can supplement your solar setup, providing power at night or during cloudy weather.

Caution

CAUTION: Be sure whatever option you choose provides a “pure sine wave” to devices that require alternating current. Ensure correct cycles-per-second, relative to your equipment’s needs. DO NOT use a “modified sine wave”. Most modern alternating-current electronics assume a pure sine wave and will be damaged if a modified sine wave is delivered. For both alternating-current and direct-current devices, be sure your option provides the correct voltage and a suitable current draw.

Author’s Path to Remote Electricity

The author has begun to explore alternatives for portable gridless electricity. To begin, he began with a small battery that can provide appropriate direct-current to the RaspberryPi for 24 hours of continuous operation. The question remains, “How to recharge the battery?” A simple wall unit allows the battery to be recharged from a standard outlet. Alternative means of recharging will be the focus of the project’s next phase of exploration.